



LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

Linux is the **foundation** of DevOps.Master the command line. **Automate everything.**Troubleshoot faster. Work like a **PRO.**

- ✓ Practical
- ✓ Production Ready
- ✓ Interview Friendly
- ✓ Time Saving



1. LINUX FILE & DIRECTORY COMMANDS



Essential commands for navigating, managing and working with files and directories.

#	COMMAND	DESCRIPTION	EXAMPLE
1	ls	List files and directories in the current directory.	ls
2	ls -l	List in long format (permissions, owner, size, date).	ls -l
3	ls -a	List all files (including hidden files).	ls -a
4	ls -lah	List all in long format (human readable).	ls -lah
5	cd /path/to/dir	Change directory to the specified path.	cd /var/log
6	cd ..	Move one directory up.	cd ..
7	pwd	Print working directory.	pwd
8	mkdir dirname	Create a new directory.	mkdir projects
9	mkdir -p /path/dir	Create parent directories if needed.	mkdir -p /opt/app/logs
10	rmdir dirname	Remove an empty directory.	rmdir old_dir
11	rm file	Remove (delete) a file.	rm file.txt
12	rm -r dir	Remove a directory and its contents.	rm -r old_project
13	rm -rf dir	Force remove a directory (careful!).	rm -rf /tmp/test
14	cp source dest	Copy file or directory.	cp file.txt /backup
15	cp -r source_dir dest_dir	Copy a directory recursively.	cp -r /app /backup
16	mv source dest	Move or rename a file or directory.	mv old.txt new.txt
17	touch filename	Create an empty file or update timestamp.	touch app.log
18	cat file	Display the contents of a file.	cat file.txt
19	less file	View file content page by page.	less file.txt
20	head -n 10 file	Show the first 10 lines of a file.	head -n 10 file.txt

EXAMPLES

List files in long format

ls -l

Create a directory

mkdir projects

Create nested directories

mkdir -p /opt/app/logs

Copy a directory

cp -r /opt/app /backup/app

View last 50 lines

tail -n 50 /var/log/syslog

Remove directory forcefully

rm -rf old_project

TIP

- Use Tab for auto-complete.
- Use '.' to move back.
- Use '*' as wildcard.
- Use '~' for home directory.
- Use '/' for root directory.
- Be careful with **rm -rf** !

QUICK REFERENCE



- .. → Current directory
- ... → Parent directory
- / → Root directory
- ~ → Home directory
- l → Long listing format
- a → Show hidden files
- h → Human readable sizes
- r → Recursive
- f → Force

COMMON FILE TYPES



- .sh → Shell script
- .conf → Config file
- .log → Log file
- .txt → Text file
- .json → JSON file
- .yaml/.yml → YAML file
- .tar → Tar archive
- .gz / .zip → Compressed file
- .sql → SQL file

PRO TIP ★

Learn to combine commands with pipes (|) and redirection (>, >>) to become a true Linux PRO!

Example:

ls -lah /var/log | grep error | tail -n 20 >> errors.log

Master these basics. They are the **foundation** of every powerful system!



LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

2. FILE SEARCH & TEXT PROCESSING COMMANDS 🔍

Powerful commands for finding files and processing text like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
21	<code>find . -name "file.txt"</code>	Search for file by name.	<code>find . -name "file.txt"</code>
22	<code>find /var -type d</code>	Find all directories.	<code>find /var -type d</code>
23	<code>find . -mtime -7</code>	Find files modified in last 7 days.	<code>find . -mtime -7</code>
24	<code>find . -size +10M</code>	Find files larger than 10MB.	<code>find . -size +10M</code>
25	<code>locate filename</code>	Quickly locate file (uses updatedb).	<code>locate app.log</code>
26	<code>which command</code>	Locate command binary.	<code>which docker</code>
27	<code>whereis command</code>	Locate binary, source and man.	<code>whereis bash</code>
28	<code>grep "error" file.log</code>	Search for pattern in file.	<code>grep "error" app.log</code>
29	<code>grep -r "error" /var/log</code>	Search recursively in directory.	<code>grep -r "error" /var/log</code>
30	<code>grep -i "error" file.log</code>	Case-insensitive search.	<code>grep -i "error" app.log</code>
31	<code>grep -v "debug" file.log</code>	Invert match (exclude).	<code>grep -v "debug" app.log</code>
32	<code>awk '{print \$1}' file</code>	Print first column.	<code>awk '{print \$1}' users.txt</code>
33	<code>awk -F: '{print \$1,\$3}' /etc/passwd</code>	Print columns with delimiter.	<code>awk -F: '{print \$1,\$3}' /etc/passwd</code>
34	<code>sed 's/old/new/g' file</code>	Replace all occurrences.	<code>sed 's/old/new/g' app.conf</code>
35	<code>sed -i 's/old/new/g' file</code>	In-place replace in file.	<code>sed -i 's/old/new/g' app.conf</code>
36	<code>sed -n '10,20p' file</code>	Print lines 10 to 20.	<code>sed -n '10,20p' server.log</code>
37	<code>cut -d',' -f1 file.csv</code>	Extract first column.	<code>cut -d',' -f1 data.csv</code>
38	<code>sort file.txt</code>	Sort lines.	<code>sort names.txt</code>
39	<code>sort -nr numbers.txt</code>	Sort numbers reverse.	<code>sort -nr load.txt</code>
40	<code>uniq file.txt</code>	Remove duplicate lines.	<code>uniq list.txt</code>

SEARCH LIKE A PRO 🎯

- Use `find` for advanced file searching.
- Use `grep` to search inside files and logs.
- Combine with `|`, `>`, `>>`, `<` for more power.
- Keep your locate database updated: `sudo updatedb`

EXAMPLES

```
# Find all .log files modified in last 2 days
find /var/log -name "*.log" -mtime -2

# Search for ERROR (case-insensitive) in all logs
grep -ri "error" /var/log

# Show top 10 largest files
find / -type f -exec du -h {} + |
sort -hr | head -n 10
```

TEXT TOOLS CHEAT

```
grep → Search text
awk → Process/Analyze
sed → Replace/Transform
cut → Extract columns
sort → Sort output
uniq → Remove duplicates
```

AWK QUICK REFERENCE 📄

- `awk '{print $0}'` → Print entire line
- `awk '{print $1}'` → Print 1st column
- `awk '{print $1,$2}'` → Print 1st and 2nd column
- `awk -F: '{print $1,$3}'` → Print 1st and 3rd column (delimiter :)
- `awk 'NR==1' file` → Print first line
- `awk 'END {print NR}' file` → Count total lines

SED QUICK REFERENCE 🖋️

- `sed 's/text/newtext/g'` → Replace text
- `sed -i 's/a/b/g' file` → In-place replace
- `sed '/pattern/d' file` → Delete matching line
- `sed '/pattern/p' file` → Print matching line
- `sed -n '5p' file` → Print line 5
- `sed -n '1,5p' file` → Print lines 1 to 5

PIPE EXAMPLES 🚰

- `ps aux | grep nginx`
- `df -h | sort -hr`
- `cat file.log | grep ERROR | wc -l`
- `netstat -tulnp | grep 80`
- `du -sh * | sort -hr | head -10`



- ☑ Use quotes in `grep` for exact match.
- ☑ Use `-i` for case-insensitive matching.
- ☑ Use `-r` for recursive search.

- ☑ Use `man <command>` to learn more.
- ☑ Combine commands with pipes (`|`) for powerful results.
- ☑ Practice these daily. Speed comes with repetition!

Small commands. Big impact. Master the basics, **automate the rest!**



LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

3. USERS, GROUPS & PERMISSIONS COMMANDS



Manage users, groups, permissions and security like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
41	id	Show user and group ID.	id
42	whoami	Show current logged in user.	whoami
43	who	Show logged in users.	who
44	w	Show who is logged in and what they are doing.	w
45	last	Show last logged in users.	last
46	lastb	Show failed login attempts.	lastb
47	adduser username	Add a new user.	sudo adduser john
48	useradd -m username	Create a new user with home directory.	sudo useradd -m john
49	passwd username	Set or change user password.	sudo passwd john
50	usermod -aG group user	Add user to group.	sudo usermod -aG docker john
51	deluser username	Delete a user.	sudo deluser john
52	usermod -r username	Delete user and home directory.	sudo userdel -r john
53	groupadd groupname	Create a new group.	sudo groupadd devops
54	groupdel groupname	Delete a group.	sudo groupdel devops
55	groups username	Show groups of a user.	groups john
56	gpasswd -a user group	Add user to group (with password).	sudo gpasswd -a john docker
57	chown owner:group file	Change owner and group of file.	sudo chown john:devops file.txt
58	chown -R owner:group dir	Change owner recursively.	sudo chown -R john:devops /opt/app
59	chmod permissions file	Change file permissions.	chmod 755 script.sh
60	chmod -R permissions dir	Change permissions recursively.	chmod -R 755 /opt/app

PERMISSION EXPLAINED



r = read (4)

w = write (2)

x = execute (1)

Example: 755

rwx

↓

owner

r-x

↓

group

r-x

↓

others

EXAMPLES

Add user and add to group

sudo adduser john

sudo usermod -aG sudo john

Change ownership recursively

sudo chown -R john:john /data

Secure sensitive file

chmod 600 config.env

SAFETY TIPS



- ✓ Always use least privilege.
- ✓ Never run everything as root.
- ✓ Use sudo for admin tasks.
- ✓ Regularly audit users and permissions.

USER MANAGEMENT FLOW

Add User
useradd -m usernameAdd to Group
usermod -aG group userSet Password
passwd usernameVerify Access
id username / groups usernameRemove User
userdel -r username

IMPORTANT FILES



- /etc/passwd → User account info
- /etc/shadow → Encrypted passwords
- /etc/group → Group info
- /etc/sudoers → Sudo access config
- /etc/gshadow → Group passwords

CHECK PERMISSIONS QUICKLY



- ls -l file.txt → Long listing
- stat file.txt → Detailed file info
- namei -l /path/to/file → Path permission trace

PRO TIPS



- ✓ Use id <user> to quickly see UID, GID and groups.
- ✓ Use chgrp group file to change group ownership.
- ✓ Use setfacl/getfacl for advanced ACL permissions.
- ✓ Use sudo -l to see allowed sudo commands.



Strong access control is the first step to a secure and reliable Linux system.





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

4. PROCESS MANAGEMENT COMMANDS

Monitor, control and manage processes like a **PRO**.

#	COMMAND	DESCRIPTION	EXAMPLE
61	<code>ps aux</code>	Show all running processes.	<code>ps aux</code>
62	<code>ps -ef</code>	Full format process list.	<code>ps -ef</code>
63	<code>ps -eo pid,ppid,cmd,%mem,%cpu</code>	Custom format process list.	<code>ps -eo pid,cmd,%mem,%cpu</code>
64	<code>top</code>	Real-time process monitoring.	<code>top</code>
65	<code>htop</code>	Interactive process viewer.	<code>htop</code>
66	<code>pstree</code>	Display processes as a tree.	<code>pstree -p</code>
67	<code>pgrep process_name</code>	Find process ID by name.	<code>pgrep nginx</code>
68	<code>pidof process_name</code>	Get PID of a process.	<code>pidof sshd</code>
69	<code>kill PID</code>	Terminate process by PID.	<code>kill 1234</code>
70	<code>kill -9 PID</code>	Force kill process.	<code>kill -9 1234</code>
71	<code>killall process_name</code>	Kill all processes by name.	<code>killall nginx</code>
72	<code>pkill process_name</code>	Kill process by name (pattern).	<code>pkill -f java</code>
73	<code>nice -n 10 command</code>	Run command with priority.	<code>nice -n 10 backup.sh</code>
74	<code>renice -n 5 -p PID</code>	Change priority of running process.	<code>renice -n 5 -p 1234</code>
75	<code>bg</code>	Resume job in background.	<code>bg</code>
76	<code>fg</code>	Bring job to foreground.	<code>fg</code>
77	<code>jobs</code>	List active jobs.	<code>jobs -l</code>
78	<code>disown -h %1</code>	Remove job from shell (keep running).	<code>disown -h %1</code>
79	<code>nohup command &</code>	Run command immune to hangup.	<code>nohup ./script.sh &</code>
80	<code>time command</code>	Measure command execution time.	<code>time ls -lah</code>

PROCESS STATES (ps aux)

- R - Running
- S - Sleeping
- D - Uninterruptible sleep
- T - Stopped
- Z - Zombie
- I - Idle

TOP SHORTCUTS

- P Sort by CPU
- M Sort by Memory
- N Sort by PID
- T Sort by Time
- k Kill process
- r Renice process
- q Quit top

EXAMPLES

```
# Show top 5 CPU consuming
ps aux --sort=-%cpu | head -n 5

# Show top 5 memory consuming
ps aux --sort=-%mem | head -n 5

# Kill process by name
pkill -f app.py

# Show process tree
pstree -p
```

UNDERSTANDING ps aux OUTPUT

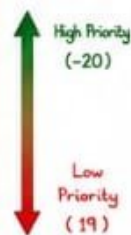
COLUMN	DESCRIPTION
USER	Owner of the process
PID	Process ID
%CPU	CPU usage
%MEM	Memory usage
VSZ	Virtual memory size
RSS	Resident memory size
TTY	Terminal
STAT	Process state
START	Start time
TIME	Total CPU time
COMMAND	Command name

COMMON USE CASES

- Find a process
`pgrep nginx`
- Kill stubborn process
`kill -9 PID`
- Monitor in real-time
`top` or `htop`
- Check resource usage
`ps aux --sort=-%cpu`
- View process hierarchy
`pstree -p`
- Measure execution time
`time command`

PROCESS PRIORITY (NICE VALUE)

Range: -20 (highest priority) to 19 (lowest priority)



- Lower (negative) nice value = higher priority
- Higher nice value = lower priority
- Root can set negative values
- Regular users can only increase (lower) priority

HTOP LEGEND

- CPU Usage
- Memory Usage
- Swap Usage

- F1 Help
- F2 Setup
- F3 Search
- F4 Filter
- F5 Tree View
- F6 Sort By
- F9 Kill
- F10 Quit

PRO TIPS



- ✓ Use `htop` instead of `top` for better visualization.
- ✓ Use `pgrep/pgrep -f` to find processes quickly.
- ✓ Use `kill -TERM` for graceful shutdown before `-9`.

- ✓ Monitor system load with `uptime` and `top`.
- ✓ Use `nice/renice` for performance tuning.
- ✓ Always check what a process does before killing!



Great systems are built by great processes. Monitor. Manage. Optimize.





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

5. SYSTEMD & SERVICES COMMANDS

Manage system services like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
81	systemctl	Control the systemd system and service manager.	systemctl
82	systemctl status service	Show status of a service.	systemctl status nginx
83	systemctl start service	Start a service.	systemctl start nginx
84	systemctl stop service	Stop a service.	systemctl stop nginx
85	systemctl restart service	Restart a service.	systemctl restart nginx
86	systemctl reload service	Reload a service configuration.	systemctl reload nginx
87	systemctl enable service	Enable service on boot.	systemctl enable nginx
88	systemctl disable service	Disable service on boot.	systemctl disable nginx
89	systemctl is-enabled service	Check if service is enabled.	systemctl is-enabled nginx
90	systemctl is-active service	Check if service is running.	systemctl is-active nginx
91	systemctl list-units --type=service	List all services.	systemctl list-units --type=service
92	systemctl list-units --failed	List failed services.	systemctl list-units --failed
93	systemctl --failed	Show failed units.	systemctl --failed
94	journalctl -u service	View logs for a service.	journalctl -u nginx
95	journalctl -xeu service	View logs with details.	journalctl -xeu nginx
96	systemctl daemon-reload	Reload systemd manager configuration.	systemctl daemon-reload
97	systemctl daemon-reexec	Re-execute systemd manager.	systemctl daemon-reexec
98	systemctl isolate target	Switch to a target (runlevel).	systemctl isolate multi-user.target
99	systemctl get-default	Show default target.	systemctl get-default
100	systemctl set-default target	Set default target.	systemctl set-default multi-user.target

COMMON TARGETS



- default.target → Default boot
- multi-user.target → Text mode
- graphical.target → GUI mode
- rescue.target → Rescue mode
- reboot.target → Reboot
- poweroff.target → Shutdown

SERVICE STATES

- active (running) → Service is running
- inactive (dead) → Service is stopped
- failed → Service failed
- activating → Service starting
- deactivating → Service stopping

EXAMPLES

```
# Check nginx status
systemctl status nginx
# Start and enable nginx
systemctl start nginx
systemctl enable nginx
# View failed services
systemctl --failed
# View nginx logs
journalctl -u nginx -f
```

TIP

- ✓ Use systemctl is-active to check service health in scripts.
- ✓ Logs are your best friend: use journalctl -xeu <service>.
- ✓ Always enable important services on boot.
- ✓ Use --now with enable to start and enable together. systemctl enable --now nginx

SYSTEMCTL SHORTCUTS

- systemctl restart → rs
- systemctl stop → st
- systemctl start → stt
- systemctl status → ss
- systemctl enable → en
- systemctl disable → dis
- systemctl reload → rl



VIEW ALL SERVICES QUICKLY

- # List all running services
systemctl list-units --type=service --state=running
- # List all enabled services
systemctl list-unit-files --type=service --state=enabled
- # List all disabled services
systemctl list-unit-files --type=service --state=disabled

JOURNALCTL QUICK REFERENCE

- journalctl → All logs
- journalctl -b → Logs since last boot
- journalctl -u service → Logs for a service
- journalctl -xe → Logs with details (errors)
- journalctl -f → Follow live logs
- journalctl --since "30 min ago" → Recent logs

SYSTEMD WORKFLOW



1. Create Service File
/etc/systemd/system/
myapp.service



2. Reload Systemd
systemctl
daemon-reload



3. Start Service
systemctl
start myapp



4. Enable on Boot
systemctl
enable myapp



5. Check Status
systemctl
status myapp



6. Monitor Logs
journalctl -u
myapp -f



Systemd is powerful. Master it once, automate everything forever!





★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

6. NETWORKING COMMANDS



Inspect, configure and troubleshoot network like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
101	ip a	Show all network interfaces.	ip a
102	ip -br a	Show interfaces in brief.	ip -br a
103	ip r	Show routing table.	ip r
104	ip r add <route>	Add a route.	ip r add 10.0.0.0/24 via 192.168.1.1
105	ip r del <route>	Delete a route.	ip r del 10.0.0.0/24
106	ip link	Show network interfaces.	ip link
107	ip link set <iface> up	Bring interface up.	ip link set eth0 up
108	ip link set <iface> down	Bring interface down.	ip link set eth0 down
109	ifconfig	Show interface configuration (legacy).	ifconfig
110	hostname -I	Show IP address of host.	hostname -I
111	hostname	Show system hostname.	hostname
112	ping <host>	Test connectivity.	ping google.com
113	ping -c 4 <host>	Send 4 ICMP packets.	ping -c 4 8.8.8.8
114	traceroute <host>	Trace route to host.	traceroute google.com
115	mtr <host>	Network diagnostic tool (real-time).	mtr google.com
116	ss -tulnp	Show listening ports and processes.	ss -tulnp
117	netstat -tulnp	Show network connections (legacy).	netstat -tulnp
118	curl <url>	Fetch URL content.	curl https://veriqta.com
119	wget <url>	Download file from URL.	wget https://veriqta.com/file.zip
120	dig <domain>	DNS lookup.	dig veriqta.com

IP ADDRESS CLASSES

- Class A: 1.0.0.0 - 126.255.255.255
- Class B: 128.0.0.0 - 191.255.255.255
- Class C: 192.0.0.0 - 223.255.255.255
- Class D: 224.0.0.0 - 239.255.255.255
- Class E: 240.0.0.0 - 255.255.255.255

COMMON PORTS

22	SSH
80	HTTP
443	HTTPS
53	DNS
3306	MySQL
5432	PostgreSQL
6379	Redis
8080	HTTP-Alt
8443	HTTPS-Alt

EXAMPLES

```
# Check IP address
ip a

# Check default gateway
ip r | grep default

# Check open ports
ss -tulnp

# Test connectivity
ping -c 4 google.com
```

TIP



- ✓ Use ip commands (modern & powerful).
- ✓ Use ss instead of netstat (faster).
- ✓ Use -c with ping to limit packets.

- ✓ Use mtr for continuous network monitoring.
- ✓ Use curl -I <url> to fetch only headers.
- ✓ Always check firewall if connection fails.



ROUTING TABLE EXAMPLE

\$ ip r

Destination	Gateway	Genmask	Interface
default	192.168.1.1	0.0.0.0	eth0
10.0.0.0	0.0.0.0	255.255.255.0	eth1
192.168.1.0	0.0.0.0	255.255.255.0	eth0

NETWORK INTERFACE STATES

- UP → Interface is active
- DOWN → Interface is inactive
- UNKNOWN → Interface status unknown
- LOWER_UP → Link is detected



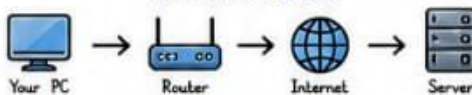
USEFUL FLAGS

- a → Show all
- r → Show routing
- t → TCP connections
- u → UDP connections
- l → Listening sockets
- n → Numeric output
- p → Show process

DNS RECORD TYPES (dig)

- A → IPv4 address
- AAAA → IPv6 address
- MX → Mail exchange
- NS → Name server
- TXT → Text record
- CNAME → Canonical name

HOW DATA TRAVELS



Use ping, traceroute, mtr to see the path.

TROUBLESHOOTING NETWORK

Can't Connect



Still not working? Check firewall, service, logs. ⚠️

★ A strong network brings systems together. A strong command keeps them running. 🏆



LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

7. SSH & REMOTE ADMINISTRATION COMMANDS



Securely connect, manage and transfer files to remote servers like a **PRO**.

#	COMMAND	DESCRIPTION	EXAMPLE
121	<code>ssh user@host</code>	Connect to a remote host.	<code>ssh ec2-user@3.10.1.25</code>
122	<code>ssh -p <port> user@host</code>	Connect using a specific port.	<code>ssh -p 2222 ubuntu@host</code>
123	<code>ssh -i <key> user@host</code>	Connect using a private key.	<code>ssh -i mykey.pem ubuntu@host</code>
124	<code>ssh -l user host</code>	Login to remote host.	<code>ssh -l devops 10.0.0.5</code>
125	<code>ssh -t user@host <cmd></code>	Execute remote command.	<code>ssh -t user@host uptime</code>
126	<code>ssh -T git@github.com</code>	Test SSH connection (no shell).	<code>ssh -T git@github.com</code>
127	<code>ssh -v user@host</code>	Verbose SSH connection.	<code>ssh -v ubuntu@host</code>
128	<code>ssh -vvv user@host</code>	More verbose (debug SSH).	<code>ssh -vvv ubuntu@host</code>
129	<code>ssh-copy-id user@host</code>	Copy your public key to host.	<code>ssh-copy-id ubuntu@host</code>
130	<code>ssh-keygen -t rsa -b 4096</code>	Generate RSA key pair.	<code>ssh-keygen -t rsa -b 4096</code>
131	<code>ssh-keygen -t ed25519</code>	Generate ed25519 key pair.	<code>ssh-keygen -t ed25519</code>
132	<code>ssh-add <key></code>	Add private key to agent.	<code>ssh-add ~/.ssh/id_rsa</code>
133	<code>ssh-add -l</code>	List keys in SSH agent.	<code>ssh-add -l</code>
134	<code>ssh-keyscan host</code>	Scan host key and add to known_hosts.	<code>ssh-keyscan 10.0.0.5 >> ~/.ssh/known_hosts</code>
135	<code>scp file user@host:/path</code>	Copy file to remote host.	<code>scp app.tar ubuntu@host:/tmp/</code>
136	<code>scp user@host:/path file</code>	Copy file from remote host.	<code>scp ubuntu@host:/tmp/app.tar .</code>
137	<code>scp -r dir user@host:/path</code>	Copy directory recursively.	<code>scp -r project/ ubuntu@host:/opt/</code>
138	<code>rsync -avz src/ user@host:/dst/</code>	Sync files to remote host.	<code>rsync -avz /data/ ubuntu@host:/backup/</code>
139	<code>sftp user@host</code>	Open SFTP interactive session.	<code>sftp ubuntu@host</code>
140	<code>exit</code>	Close SSH/SFTP session.	<code>exit</code>

SSH CONNECTION OPTIONS



- `-p <port>` → Use custom port
- `-i <key>` → Use identity file
- `-L <user>` → Specify user
- `-t` → Force pseudo-terminal
- `-T` → Disable pseudo-terminal
- `-v` → Verbose output
- `-vvv` → Debug mode

EXAMPLES



```
# Connect to server
ssh ubuntu@10.0.0.5

# Connect using key & port
ssh -i key.pem -p 2222 ubuntu@host

# Run command on remote host
ssh -t ubuntu@host "df -h"

# Copy file to remote
scp app.zip ubuntu@host:/tmp/

# Sync directory
rsync -avz /data/ ubuntu@host:/backup/
```

SECURITY TIPS



- ☒ Use SSH key authentication.
- ☒ Disable root login.
- ☒ Change default SSH port.
- ☒ Use strong passphrases.
- ☒ Keep your private key safe.
- ☒ Regularly audit
`~/.ssh/authorized_keys`

SSH CONFIG FILE (~/.ssh/config)



Create shortcuts for servers.

```
Host prod
  HostName 3.10.1.25
  User ubuntu
  Port 2222
  IdentityFile ~/.ssh/prod.pem
```

```
Host dev
  HostName 10.0.0.5
  User ec2-user
  IdentityFile ~/.ssh/dev.pem
```

```
Usage:
ssh prod
ssh dev
```

SFTP BASIC COMMANDS



COMMAND	DESCRIPTION
<code>lcd <localdir></code>	Change local directory
<code>cd <remotedir></code>	Change remote directory
<code>ls</code>	List remote files
<code>lls</code>	List local files
<code>get <file></code>	Download file
<code>put <file></code>	Upload file
<code>rm <file></code>	Delete remote file
<code>bye / exit</code>	Exit SFTP

COMMON SSH CONFIG LOCATIONS



- User config: `~/.ssh/config`
- Private keys: `~/.ssh/id_rsa`
- Public keys: `~/.ssh/id_rsa.pub`
- Known hosts: `~/.ssh/known_hosts`
- Authorized keys (server):
`/home/<user>/.ssh/authorized_keys`

REMOTE ADMIN BEST PRACTICES



- ☒ Use keys, not passwords.
- ☒ Enable 2FA (where possible).
- ☒ Limit access with firewall.
- ☒ Monitor login attempts.
- ☒ Keep SSH software updated.

SSH WORKFLOW



1. Generate Key
`ssh-keygen`



2. Copy Key
`ssh-copy-id user@host`



3. Connect
`ssh user@host`



4. Run Commands
`uptime, df -h, etc`



5. Transfer Files
`scp / rsync / sftp`



6. Secure & Monitor
Logs, alerts, audit



Master SSH and you can control servers, automate tasks and scale like a **PRO!**





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

8. DISK, STORAGE & FILESYSTEM COMMANDS

Manage disks, partitions, filesystems and storage like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
141	df -h	Show disk space usage (human readable).	df -h
142	df -i	Show inode usage.	df -i
143	du -sh <dir>	Show directory size.	du -sh /var/log
144	du -ah <dir> sort -h	Show directory sizes (sorted).	du -ah /var sort -h
145	lsblk	List block devices.	lsblk
146	lsblk -f	List block devices with filesystem.	lsblk -f
147	fdisk -l	List all partitions.	fdisk -l
148	parted -l	List partitions (parted).	parted -l
149	blkid	Show block device attributes.	blkid
150	mkfs.ext4 <device>	Create ext4 filesystem.	mkfs.ext4 /dev/sdb1
151	mkfs.xfs <device>	Create XFS filesystem.	mkfs.xfs /dev/sdb1
152	mkfs -t ext4 <device>	Create filesystem (generic).	mkfs -t ext4 /dev/sdb1
153	mount <device> <dir>	Mount filesystem.	mount /dev/sdb1 /mnt
154	mount -a	Mount all from /etc/fstab.	mount -a
155	umount <dir>	Unmount filesystem.	umount /mnt
156	findmnt	Show mounted filesystems.	findmnt
157	cat /etc/fstab	Show filesystem mount config.	cat /etc/fstab
158	fsck <device>	Check filesystem.	fsck /dev/sdb1
159	fsck -y <device>	Auto-fix filesystem.	fsck -y /dev/sdb1
160	smartctl -a <device>	Check disk health (SMART).	smartctl -a /dev/sda

COMMON DIRECTORIES



- / → Root filesystem
- /boot → Boot files
- /etc → System config
- /var → Variable data
- /tmp → Temporary files
- /home → User home dirs
- /mnt → Mount point
- /media → Removable media
- /opt → Optional software

COMMON DEVICE NAMES

- /dev/sda → First SATA/SCSI disk
- /dev/sdb → Second disk
- /dev/sdc → Third disk
- /dev/nvme0n1 → NVMe disk
- /dev/sda1 → First partition
- /dev/sda2 → Second partition
- /dev/mapper/* → LVM devices

EXAMPLES



- # Check disk usage
df -h
- # Find big directories in /var
du -ah /var | sort -h | tail -n 10
- # Mount a disk
mount /dev/sdb1 /data
- # Auto mount on boot
echo "/dev/sdb1 /data ext4
defaults 0 2" >> /etc/fstab

PRO TIPS



- ✓ Always check disk before mounting.
- ✓ Use UUID in /etc/fstab for stability.
- ✓ Run fsck only on unmounted partitions.
- ✓ Monitor disk health with smartctl.
- ✓ Use df -h to monitor disk usage.

DSK USAGE SUMMARY



Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	20G	28G	42%	/
/dev/sda2	100G	60G	38G	62%	/var
tmpfs	2G	0	2G	0%	/run

FILESYSTEM TYPES



- ext4 → Linux default (stable)
- xfs → High performance (large files)
- btrfs → Copy-on-write, snapshots
- vfat → For FAT32/Windows
- ntfs → For NTFS (Windows)
- swap → Swap memory area

LSBLK OUTPUT EXAMPLE

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	120G	0	disk	
└sda1	8:1	0	500M	0	part	/boot
└sda2	8:2	0	50G	0	part	/
└sda3	8:3	0	69.5G	0	part	
└vg-root	253:0	0	69.5G	0	lvm	/var

MOUNT OPTIONS (COMMON)

- defaults → Read/write, suid, dev, exec, auto, nouser, noexec
- ro → Mount read-only
- rw → Mount read-write
- noexec → Disable program execution
- nosuid → Disable set-user-id programs
- nodelv → Ignore device files
- auto → Mount at boot
- noauto → Don't mount at boot

DISK MANAGEMENT WORKFLOW



1. Check Disk
lsblk / fdisk -l
2. Create Partition
fdisk / parted
3. Create FS
mkfs.ext4
4. Mount Disk
mount /dev/sdb1 /data
5. Add to fstab
echo >> /etc/fstab

CHECK DISK HEALTH



- ✓ smartctl -a /dev/sda
- ✓ Look for "PASSED" status
- ✓ Check reallocated sectors
- ✓ Monitor temperature
- ✓ Schedule regular checks



A healthy disk is a happy system. Monitor. Maintain. Backup. Always!





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

9. LOGS & JOURNALCTL COMMANDS

View, filter and analyze system logs like a **PRO**.

#	COMMAND	DESCRIPTION	EXAMPLE
161	<code>journalctl</code>	View all logs.	<code>journalctl</code>
162	<code>journalctl -f</code>	Follow logs in real-time.	<code>journalctl -f</code>
163	<code>journalctl -n 50</code>	Show last 50 log lines.	<code>journalctl -n 50</code>
164	<code>journalctl -p err</code>	Show error logs only.	<code>journalctl -p err</code>
165	<code>journalctl -p warning</code>	Show warning and above.	<code>journalctl -p warning</code>
166	<code>journalctl -u nginx</code>	Show logs for nginx service.	<code>journalctl -u nginx</code>
167	<code>journalctl -u docker -f</code>	Follow docker service logs.	<code>journalctl -u docker -f</code>
168	<code>journalctl --since "1 hour ago"</code>	Logs from 1 hour ago.	<code>journalctl --since "1 hour ago"</code>
169	<code>journalctl --since "2024-01-01"</code>	Logs since a specific date.	<code>journalctl --since "2024-01-01"</code>
170	<code>journalctl --until "1 hour ago"</code>	Logs until 1 hour ago.	<code>journalctl --until "1 hour ago"</code>
171	<code>journalctl --since today</code>	Logs from today.	<code>journalctl --since today</code>
172	<code>journalctl -o short</code>	Short output format.	<code>journalctl -o short</code>
173	<code>journalctl -o json-pretty</code>	JSON formatted output.	<code>journalctl -o json-pretty</code>
174	<code>journalctl -k</code>	Show kernel logs.	<code>journalctl -k</code>
175	<code>journalctl -b</code>	Logs from current boot.	<code>journalctl -b</code>
176	<code>journalctl -xb</code>	Logs from previous boot.	<code>journalctl -xb</code>
177	<code>journalctl --disk-usage</code>	Show journal disk usage.	<code>journalctl --disk-usage</code>
178	<code>journalctl --vacuum-time=7d</code>	Remove logs older than 7 days.	<code>journalctl --vacuum-time=7d</code>
179	<code>journalctl --vacuum-size=500M</code>	Limit journal size to 500MB.	<code>journalctl --vacuum-size=500M</code>
180	<code>journalctl _PID=1234</code>	Show logs for specific PID.	<code>journalctl _PID=1234</code>

LOG PRIORITY LEVELS ⚠

- 0 - emerg (system is unusable)
- 1 - alert (action must be taken)
- 2 - crit (critical conditions)
- 3 - err (error conditions)
- 4 - warning (warning conditions)
- 5 - notice (normal but significant)
- 6 - info (informational)
- 7 - debug (debug-level messages)

COMMON UNITS (-u)

- sshd → SSH service
- nginx → Nginx web server
- docker → Docker service
- kubelet → Kubernetes node agent
- networkd → NetworkManager
- cron → Cron daemon
- systemd → System manager
- firewalld → Firewall service

EXAMPLES 💡

```
# Follow system logs
journalctl -f

# Show errors
journalctl -p err -n 100

# Nginx logs since today
journalctl -u nginx --since today

# Logs in JSON
journalctl -u docker -o json-pretty

# Clean old logs (7 days)
journalctl --vacuum-time=7d
```

TIP



- ✓ Use `-f` to monitor logs in real-time.
- ✓ Use `-u <service>` to filter by service.
- ✓ Regularly clean older logs to save disk space.

- ✓ Use `-p` to focus on important logs.
- ✓ Use `--since/--until` for time range analysis.



JOURNAL WORKFLOW

1. View all logs
`journalctl`
2. Filter by service
`journalctl -u nginx`
3. Set time range
`--since today`
4. Follow in real-time
`-f`
5. Analyze & troubleshoot

FILTER BY TIME EXAMPLES

- `--since "10 minutes ago"`
- `--since "1 hour ago"`
- `--since "yesterday"`
- `--since "2024-05-01 10:00:00"`
- `--until "2024-05-01 12:00:00"`
- `--since today`
- `--since boot`

OUTPUT FORMATS 📄

- `-o short` → Default short
- `-o short-precise` → With microseconds
- `-o verbose` → More details
- `-o json` → JSON format
- `-o json-pretty` → Pretty JSON
- `-o cat` → Raw message

HANDY ALIASES

```
alias jl='journalctl -f'
alias jn='journalctl -n 100'
alias je='journalctl -p err -n 100'
alias js='journalctl -u'
alias jb='journalctl -b'
```

SYSTEM LOG LOCATIONS (LEGACY FILES)

On some systems, logs are also stored in files.

- `/var/log/syslog` → General system logs
- `/var/log/auth.log` → Authentication logs
- `/var/log/messages` → Kernel & system messages
- `/var/log/secure` → Security logs (RHEL/CentOS)
- `/var/log/dmseg` → Kernel ring buffer

COMPARE: `journalctl` vs `cat`

- | | | |
|----------------------------------|----|----------------------------|
| <code>journalctl</code> (Better) | VS | <code>cat</code> (Old Way) |
| ✓ Structured logs | | ✗ Unstructured |
| ✓ Powerful filtering | | ✗ Hard to filter |
| ✓ Time based queries | | ✗ No time range |
| ✓ Service based | | ✗ File based only |
| ✓ Easy to read | | ✗ Hard to analyze |

BEST PRACTICES ★

- ✓ Monitor logs regularly.
- ✓ Set up log rotation.
- ✓ Alert on critical errors.
- ✓ Centralize logs (ELK/EFK).
- ✓ Keep logs for compliance.

★ Logs tell you what happened. Monitoring tells you what is happening. Act before users notice! 🏆





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

10. PROCESS MANAGEMENT COMMANDS

Monitor, control and manage processes like a **PRO**.

#	COMMAND	DESCRIPTION	EXAMPLE
181	ps aux	Show all running processes.	ps aux
182	ps -ef	Full format process listing.	ps -ef
183	top	Real-time process viewer.	top
184	htop	Interactive process viewer.	htop
185	pidof <name>	Find PID of a process.	pidof nginx
186	pgrep <name>	Find PID(s) using pattern.	pgrep java
187	pgrep -f <pattern>	Find PID(s) by full command line.	pgrep -f python
188	pstree	Show process tree.	pstree
189	pstree -p	Show process tree with PIDs.	pstree -p
190	kill <PID>	Terminate process by PID.	kill 1234
191	kill -9 <PID>	Force kill process (SIGKILL).	kill -9 1234
192	killall <name>	Kill all processes by name.	killall nginx
193	pkill <pattern>	Kill processes by pattern.	pkill -f python
194	nice -n 10 <cmd>	Run command with priority.	nice -n 10 tar -czf bk.tar.gz /data
195	renice -n 5 -p <PID>	Change process priority.	renice -n 5 -p 1234
196	nohup <cmd> &	Run command immune to hangup.	nohup long.sh &
197	bg	List background jobs.	bg
198	fg %1	Bring job to foreground.	fg %1
199	jobs	List active jobs.	jobs
200	wait	Wait for a background job to finish.	wait

PROCESS STATES



- R Running
- S Sleeping (interruptible)
- D Uninterruptible sleep
- T Stopped
- Z Zombie
- I Idle

COMMON SIGNALS



- 1 (SIGHUP) → Hangup
- 2 (SIGINT) → Interrupt
- 9 (SIGKILL) → Kill (force)
- 15 (SIGTERM) → Terminate
- 18 (SIGCONT) → Continue
- 19 (SIGSTOP) → Stop

Use 'kill -l' to see all signals

EXAMPLES



- # Show top 5 CPU consuming processes
ps aux --sort=-%cpu | head -n 6
- # Kill all python processes
pkill -f python
- # Run command in background
nohup python app.py &
- # Change priority of process
renice -n -5 -p 1234

TIP



- ✓ Use htop for better visualization.
- ✓ Use pgrep -f to match full command lines.

- ✓ Avoid kill -9 unless necessary.
- ✓ Monitor critical processes regularly.



htop SHORTCUTS



- F1 → Help
- F2 → Setup
- F3 → Search
- F4 → Filter
- F5 → Tree view
- F6 → Sort
- F9 → Kill process
- F10 → Quit

ps COMMON OPTIONS



- ps -e → All processes
- ps -f → Full format
- ps -u user → Processes by user
- ps -p PID → Specific PID
- ps -o pid,ppid,cmd,%cpu,%mem → Custom output

RESOURCE MONITORING TOOLS



- top → CPU, Memory, Tasks
- htop → Enhanced top
- atop → Advanced system monitor
- glances → All-in-one monitoring
- nmon → Performance monitor
- iotop → I/O usage by process

PROCESS PRIORITY (NICE VALUES)



- 20 0 +19
- High Priority Normal Low Priority
- Lower value = Higher priority
 - Use 'nice' to start a process with priority
 - Use 'renice' to change priority of running process

USEFUL TIPS



- ✓ Keep critical services running.
- ✓ Automate process restarts with systemd.
- ✓ Use monitoring + alerts.
- ✓ Document important PIDs.
- ✓ Don't forget resource limits (ulimit).

COMMON PROCESS DIRECTORIES



- /proc → Process information (live)
- /var/run → PID files
- /etc/systemd/system → Service configs
- /etc/init.d → Init scripts (legacy)



A good engineer monitors processes. A great engineer prevents problems.





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

11. USER & GROUP MANAGEMENT COMMANDS

Manage users, groups and permissions like a **PRO**.

#	COMMAND	DESCRIPTION	EXAMPLE
201	<code>id <user></code>	Show user and group IDs.	<code>id devops</code>
202	<code>whoami</code>	Show current logged in user.	<code>whoami</code>
203	<code>who</code>	Show logged in users.	<code>who</code>
204	<code>w</code>	Show who is logged in and activity.	<code>w</code>
205	<code>last</code>	Show last logged in users.	<code>last</code>
206	<code>users</code>	Show currently logged in users.	<code>users</code>
207	<code>adduser <user></code>	Add a new user.	<code>sudo adduser devops</code>
208	<code>useradd <user></code>	Add user (advanced).	<code>sudo useradd devops</code>
209	<code>passwd <user></code>	Set/change user password.	<code>passwd devops</code>
210	<code>usermod <options> <user></code>	Modify user account.	<code>usermod -aG sudo devops</code>
211	<code>userdel <user></code>	Delete a user.	<code>sudo userdel devops</code>
212	<code>deluser <user></code>	Delete a user (Debian/Ubuntu).	<code>sudo deluser devops</code>
213	<code>groupadd <group></code>	Create a new group.	<code>sudo groupadd devops</code>
214	<code>groupdel <group></code>	Delete a group.	<code>sudo groupdel devops</code>
215	<code>groups <user></code>	Show groups of user.	<code>groups devops</code>
216	<code>gpasswd -a <user> <group></code>	Add user to group.	<code>sudo gpasswd -a devops sudo</code>
217	<code>gpasswd -d <user> <group></code>	Remove user from group.	<code>sudo gpasswd -d devops sudo</code>
218	<code>chown <user>:<group> <file></code>	Change file owner and group.	<code>sudo chown devops:devops file.txt</code>
219	<code>chgrp <group> <file></code>	Change group of file.	<code>sudo chgrp devops file.txt</code>
220	<code>chmod <mode> <file></code>	Change file permissions.	<code>chmod 755 script.sh</code>

COMMON USER FILES



- `/etc/passwd` → User account info
- `/etc/shadow` → Encrypted passwords
- `/etc/group` → Group info
- `/etc/gshadow` → Secure group info
- `/etc/sudoers` → Sudo access control

USER TYPES



- `root` → Super user
- System user → Service accounts
- Normal user → Human users
- Service user → Daemons/Apps

EXAMPLES



Add user and set password
`sudo adduser john`
`sudo passwd john`

Add user to sudo group
`sudo usermod -aG sudo john`

Change ownership recursively
`sudo chown -R john:john /data`

Set permission
`chmod 644 file.txt`

TIP



- ✓ Use `adduser` on Ubuntu/Debian.
- ✓ Use `useradd` on RHEL/CentOS.
- ✓ Always use strong passwords.

- ✓ Don't edit `/etc/passwd` manually.
- ✓ Use sudo access instead of root login.
- ✓ Principle of least privilege is key.



FILE PERMISSIONS (RECAP)

Format: `rwx r-x r-x (755)`

Digit	Permission	Meaning
4	r (read)	Read
2	w (write)	Write
1	x (execute)	Execute
Owner	Group	Others
7	5	5

→ `rwx-r-x-r-x`

OWNERSHIP EXAMPLES



- `chown user file.txt` → Change owner
- `chown user:group file.txt` → Change owner & group
- `chown -R user:group /dir` → Change recursively
- `chgrp group file.txt` → Change group only

USERMOD COMMON OPTIONS



- `-aG group` → Add to group
- `-G group` → Set primary group
- `-l newname` → Change username
- `-d /home/user` → Change home directory
- `-s /bin/bash` → Change default shell
- `-L` → Lock user account
- `-U` → Unlock user account

SUDO ACCESS CONTROL



- `sudo` allows admin privileges.
- User must be in `sudo` group (Ubuntu) or `wheel` group (RHEL).
- Edit `sudoers` safely using: `sudo visudo`

⚠ Never edit `/etc/sudoers` directly!

USER MANAGEMENT WORKFLOW



1. Add User
`adduser user`
2. Add to Group
`usermod -aG`
3. Set Password
`passwd user`
4. Set Permissions
`chown/chmod`
5. Verify Access
`su - user`

BEST PRACTICES



- ✓ Create non-root users.
- ✓ Use `sudo`, not `root`.
- ✓ Review user access regularly.
- ✓ Remove unused accounts.
- ✓ Enforce strong password policy.
- ✓ Audit user and group changes.



Good access control isn't about blocking users; it's about giving the right access to the right users.





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

12. NETWORKING & DIAGNOSTIC COMMANDS



Troubleshoot network issues and diagnose connectivity like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
221	<code>ip a</code>	Show all network interfaces.	<code>ip a</code>
222	<code>ip addr show</code>	Show IP addresses.	<code>ip addr show</code>
223	<code>ip r</code>	Show routing table.	<code>ip r</code>
224	<code>ss -tulnp</code>	Show listening ports and services.	<code>ss -tulnp</code>
225	<code>ss -tulnp grep :80</code>	Check service on port 80.	<code>ss -tulnp grep :80</code>
226	<code>netstat -tulnp</code>	Legacy: Show open ports.	<code>netstat -tulnp</code>
227	<code>ping <host></code>	Check connectivity to host.	<code>ping google.com</code>
228	<code>ping -c 4 <host></code>	Ping host 4 times.	<code>ping -c 4 8.8.8.8</code>
229	<code>traceroute <host></code>	Trace route to host.	<code>traceroute google.com</code>
230	<code>mtr <host></code>	Network diagnostic (combines ping & traceroute).	<code>mtr google.com</code>
231	<code>dig <domain></code>	DNS lookup.	<code>dig veriqta.com</code>
232	<code>nslookup <domain></code>	DNS lookup (legacy).	<code>nslookup veriqta.com</code>
233	<code>host <domain></code>	DNS lookup.	<code>host veriqta.com</code>
234	<code>curl <url></code>	Transfer data from URL.	<code>curl -I https://veriqta.com</code>
235	<code>wget <url></code>	Download file from URL.	<code>wget https://veriqta.com/file</code>
236	<code>nc -zv <host> <port></code>	Check if port is open.	<code>nc -zv google.com 80</code>
237	<code>nc -l <port></code>	Listen on a port.	<code>nc -l 4444</code>
238	<code>tcpdump -i any</code>	Capture packets.	<code>tcpdump -i any port 80</code>
239	<code>iftop</code>	Monitor bandwidth usage.	<code>iftop</code>
240	<code>ethtool <iface></code>	Show network interface stats.	<code>ethtool eth0</code>

COMMON INTERFACES



- `eth0` → Ethernet
- `ens33` → Modern Ethernet
- `enp0s3` → PCI Ethernet
- `lo` → Loopback
- `wlan0` → Wi-Fi

COMMON PORTS



- 22 → SSH
- 80 → HTTP
- 443 → HTTPS
- 53 → DNS
- 25 → SMTP
- 3306 → MySQL
- 5432 → PostgreSQL

EXAMPLES



- # Check IP address
`ip a`
- # Check default gateway
`ip r | grep default`
- # Check listening services
`ss -tulnp`
- # Ping and trace route
`ping -c 4 8.8.8.8`
`traceroute 8.8.8.8`
- # Check port 22 on server
`nc -zv server 22`

TIP



- ✓ Use `ip a` instead of `ifconfig` (deprecated).
- ✓ Use `ss` instead of `netstat` (faster & modern).
- ✓ Use `-c` to limit ping count.

- ✓ Use `nc` (netcat) for quick connectivity tests.
- ✓ Use `dig` for advanced DNS lookups.



IP ADDRESS TYPES

- 127.0.0.1 → Loopback (localhost)
- Private IP Ranges:
 - 10.0.0.0/8
 - 172.16.0.0/12
 - 192.168.0.0/16
- Public IP → Internet-routable

ROUTE TYPES (`ip r`)

- default via 192.168.1.1 dev eth0
- 10.0.0.0/24 dev eth0
- 192.168.1.0/24 dev wlan0
- default → Default gateway

NETWORK TROUBLESHOOT FLOW



1. Check IP
`ip a`
2. Check DNS
`dig google.com`
3. Check Route
`ip r`
4. Check Port
`ss -tulnp`

USEFUL CURL EXAMPLES

- `curl -I https://veriqta.com` → HTTP headers
- `curl -L https://veriqta.com` → Follow redirects
- `curl -o file.html https://veriqta.com` → Save output
- `curl -X POST -d "data=1" http://api` → Send data

PING & TRACEROUTE EXAMPLES

- `ping -c 4 google.com` → Ping 4 times
- `ping -i 2 google.com` → Interval 2 sec
- `traceroute google.com` → Trace route
- `mtr -rw google.com` → Continuous report

DIAGNOSTIC TOOLS



- `iftop` → Real-time bandwidth monitor
- `nload` → Network load monitor
- `bmon` → Bandwidth monitor
- `wireshark` → Packet analyzer (GUI)

FIREWALL CHECK



- `sudo ufw status` → UFW firewall
- `sudo firewall-cmd --list-all` → Firewall zones
- `iptables -L -n -v` → Iptables rules

DNS RECORD TYPES (`dig`)

- `dig veriqta.com A` → IPv4 record
- `dig veriqta.com AAAA` → IPv6 record
- `dig veriqta.com MX` → Mail exchange
- `dig veriqta.com NS` → Name server

BEST PRACTICES



- ✓ Always verify IP, DNS and route first.
- ✓ Understand which ports are open.
- ✓ Monitor network performance.
- ✓ Secure unnecessary open ports.



A strong network is the backbone of a stable system. Monitor. Diagnose. Secure!



FOLLOW @VERIQTA FOR MORE DEVOPS CONTENT



LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

13. SYSTEM PERFORMANCE & MONITORING COMMANDS

Monitor system performance and resource usage like a **PRO**.

#	COMMAND	DESCRIPTION	EXAMPLE
241	top	Real-time system monitor.	top
242	htop	Interactive process viewer.	htop
243	atop	Advanced system & process monitor.	atop
244	glances	Cross-platform monitoring tool.	glances
245	vmstat	Virtual memory statistics.	vmstat 1 5
246	iostat	I/O statistics.	iostat -xz 1 5
247	sar	System activity reporter.	sar -u 1 5
248	mpstat	CPU usage per processor.	mpstat -P ALL 1 5
249	pidstat	Per-process statistics.	pidstat -u 1 5
250	uptime	System uptime and load average.	uptime
251	w	Who is logged in and what they are doing.	w
252	ps aux	Snapshot of running processes.	ps aux
253	ps -ef	Full format process listing.	ps -ef
254	free -h	Memory usage summary.	free -h
255	df -h	Disk space usage.	df -h
256	du -sh *	Disk usage of files/folders.	du -sh /var/*
257	dstat	Versatile resource statistics.	dstat -cdngy
258	nmon	Performance monitoring tool.	nmon
259	iotop	Top I/O consuming processes.	iotop
260	iftop	Real-time network bandwidth usage.	iftop -i eth0

TOP COMMANDS SUMMARY



- top / htop → Process & CPU
- free → Memory
- df → Disk
- iostat / dstat → I/O
- iftop → Network
- vmstat / sar → System stats

CPU STATES (%us, %sy, %id)



- %us → User time
- %sy → System time
- %id → Idle time
- %wa → I/O wait
- %hi → Hardware IRQ
- %si → Software IRQ
- %st → Steal time

EXAMPLES



- # Check CPU & memory
top
- # Check memory
free -h
- # Check disk usage
df -h
- # Check I/O performance
iostat -xz 1 5
- # Check network usage
iftop -i eth0

TIP



- ✓ Use htop for better visualization than top.
- ✓ Use dstat for quick all-in-one stats.

- ✓ Use iotop to find disk-hungry processes.

- ✓ Run monitoring tools in screen or tmux for 24/7 monitoring.



MEMORY USAGE (free -h)



Column	Meaning
total	Total RAM
used	Used RAM
free	Free RAM
shared	Shared memory
buff/cache	Cache & buffers
available	Available RAM

LOAD AVERAGE (uptime)



Example: 15:42:01 up 5 days, 3:10, 2 users, load average: 0.35, 0.42, 0.38

- 1 min → Short-term load
- 5 min → Medium-term load
- 15 min → Long-term load
- Lower is better (< CPU cores)

DISK I/O (iostat)



Column	Meaning
r/s	Reads per second
w/s	Writes per second
rkB/s	Read KB per second
wkB/s	Write KB per second
%util	Device utilization %

PROCESS STATES (ps)

- R Running
- S Sleeping
- D Uninterruptible sleep
- Z Zombie
- T Stopped
- I Idle (Kernel)



MONITORING TOOLS COMPARISON

Tool	Type	Best For
top	Basic	Quick overview
htop	Interactive	Detailed process view
atop	Advanced	Historical state
glances	All-in-one	Easy monitoring
dstat	All-in-one	DevOps dashboards

RESOURCE UTILIZATION GUIDELINES



- CPU usage > 80% consistently → Investigate
- Memory usage > 90% → Optimize or add RAM
- Disk usage > 85% → Clean or expand
- Load average > CPU cores → System overloaded
- I/O wait > 20% → Check disk performance
- Network usage high → Check bandwidth hogs

PERSISTENT MONITORING



- ✓ Use screen or tmux
- ✓ Run tools in background
- ✓ Store logs for historical analysis
- ✓ Set up alerts with Grafana/Prometheus
- ✓ Automate with cron jobs

PERFORMANCE MONITORING WORKFLOW



1. Monitor
top / htop



2. Analyze
vmstat / sar



3. Diagnose
iostat / iotop



4. Check Network
iftop / nload



5. Take Action
Optimize / Scale



6. Monitor Again
Repeat cycle



Monitor actively. Analyze deeply. Optimize smartly.



FOLLOW @VERIQTAT FOR MORE DEVOPS CONTENT





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

14. STORAGE & FILESYSTEM COMMANDS



Manage disks, partitions, filesystems and disk usage like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
261	<code>df -h</code>	Disk space usage (human readable).	<code>df -h</code>
262	<code>du -sh <dir></code>	Disk usage of a directory.	<code>du -sh /var</code>
263	<code>du -ah <dir> sort -h</code>	Detailed disk usage sorted.	<code>du -ah /home sort -h</code>
264	<code>lsblk</code>	List block devices.	<code>lsblk</code>
265	<code>fdisk -l</code>	Show all disk partitions.	<code>fdisk -l</code>
266	<code>parted -l</code>	Print partition table.	<code>parted -l</code>
267	<code>blkid</code>	Show block device attributes.	<code>blkid</code>
268	<code>mkfs.ext4 /dev/sdX1</code>	Create ext4 filesystem.	<code>mkfs.ext4 /dev/sdb1</code>
269	<code>mkfs.xfs /dev/sdX1</code>	Create xfs filesystem.	<code>mkfs.xfs /dev/sdb1</code>
270	<code>mount /dev/sdX1 /mnt</code>	Mount a filesystem.	<code>mount /dev/sdb1 /mnt</code>
271	<code>umount /mnt</code>	Unmount filesystem.	<code>umount /mnt</code>
272	<code>mount column -t</code>	Show mounted filesystems.	<code>mount column -t</code>
273	<code>fstab -l</code>	List entries in fstab.	<code>fstab -l</code>
274	<code>swapon --show</code>	Show swap usage.	<code>swapon --show</code>
275	<code>dd if=/dev/zero of=file bs=1M</code>	Create empty file (test disk).	<code>dd if=/dev/zero of=testing.txt bs=1M count=100</code>
276	<code>rsync -ah --progress src/ dest/</code>	Sync files/directories.	<code>rsync -ah --progress /data/ /backup/</code>
277	<code>tar -czvf backup.tar.gz /data</code>	Create compressed archive.	<code>tar -czvf backup.tar.gz /data</code>
278	<code>tar -xzf backup.tar.gz</code>	Extract compressed archive.	<code>tar -xzf backup.tar.gz</code>
279	<code>find / -type f -size +100M</code>	Find files larger than 100MB.	<code>find / -type f -size +100M</code>
280	<code>locate filename</code>	Find file using database.	<code>locate nginx.conf</code>

DISK USAGE SUMMARY



- `df -h` → Overall filesystem usage
- `du -sh` → Directory size summary
- `du -ah | sort -h` → Detailed usage
- `ncdu` → Interactive disk usage

COMMON MOUNT OPTIONS



- `ro` → Read only
- `rw` → Read write
- `auto` → Mount automatically
- `noauto` → Don't mount automatically
- `exec` → Allow program execution
- `noexec` → Disallow program execution
- `user` → Allow user mount
- `defaults` → Default options

EXAMPLES



- # Check disk usage
`df -h`
- # Find big directories
`du -h --max-depth=1 / | sort -hr`
- # Mount and check
`mount /dev/sdb1 /mnt`
`df -h | grep mnt`
- # Backup /data
`tar -czvf data_backup.tar.gz /data`

TIP



- ✓ Use `df -h` to quickly check disk usage.
- ✓ Use `du` carefully on large directories.
- ✓ Always backup data before formatting disks.

- ✓ Use `blkid` before mounting unknown devices.
- ✓ Keep `fstab` updated for permanent mounts.



PARTITION COMMANDS



- `fdisk -l` → List partitions (MBR/GPT)
- `gdisk -l` → GPT partition table
- `cfdisk /dev/sdX` → Partition editor
- `partprobe` → Reread partition table
- `lsblk -f` → List with filesystem info

FILESYSTEM TYPES



- `ext4` → Standard Linux filesystem
- `xfs` → High performance filesystem
- `btrfs` → Copy-on-write, snapshots
- `swap` → Swap space
- `vfat` → FAT filesystem (USB/SD)

USEFUL FIND EXAMPLES



- `find / -name "*.log"` → Find log files
- `find / -type d -empty` → Find empty dirs
- `find / -mtime -7` → Modified in last 7 days
- `find / -user root` → Files owned by root
- `find / -perm 644` → Find files with 644 perms

DD COMMAND EXAMPLES

- `dd if=/dev/sda of=/dev/sdb` → Disk clone
- `dd if=file.iso of=/dev/sdb` → Write ISO to disk
- `dd if=/dev/sda | gzip > img.gz` → Backup image

RSYNC COMMON OPTIONS

- `-a` → Archive mode (preserve everything)
- `-v` → Verbose
- `-z` → Compress data
- `--delete` → Delete extraneous files

MOUNT TROUBLESHOOTING



- mount: wrong fs type → Check filesystem type
- device is busy → Use `lsof | grep mountpoint`
- permission denied → Check permissions / fstab
- read-only filesystem → Check mount options & disk

STORAGE MANAGEMENT WORKFLOW



Disks don't fail suddenly. They warn you. Monitor. Backup. Repeat.



FOLLOW @VERIQTAT FOR MORE DEVOPS CONTENT





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

15. LOG MANAGEMENT COMMANDS

Collect, analyze and manage logs like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
281	<code>journalctl</code>	View and query systemd journal logs.	<code>journalctl</code>
282	<code>journalctl -xe</code>	Show journal logs with errors.	<code>journalctl -xe</code>
283	<code>journalctl -u <service></code>	Show logs for a specific service.	<code>journalctl -u nginx</code>
284	<code>journalctl -f</code>	Follow real-time journal logs.	<code>journalctl -f</code>
285	<code>journalctl -S "-1 hour"</code>	Show logs from last 1 hour.	<code>journalctl -S "-1 hour"</code>
286	<code>journalctl -p err</code>	Show only error logs.	<code>journalctl -p err</code>
287	<code>tail -f <file></code>	Follow log file in real-time.	<code>tail -f /var/log/syslog</code>
288	<code>tail -n 100 <file></code>	Show last 100 lines.	<code>tail -n 100 /var/log/nginx/access.log</code>
289	<code>cat <file></code>	Display entire file content.	<code>cat /var/log/dmesg</code>
290	<code>less <file></code>	Open file with pager.	<code>less /var/log/syslog</code>
291	<code>grep "error" <file></code>	Search for "error" in file.	<code>grep "error" app.log</code>
292	<code>grep -i "fail" <file></code>	Case-insensitive search for "fail".	<code>grep -i "fail" app.log</code>
293	<code>awk '{print \$1}' <file></code>	Print first column from file.	<code>awk '{print \$1}' access.log</code>
294	<code>sed -n '1,10p' <file></code>	Print first 10 lines of file.	<code>sed -n '1,10p' app.log</code>
295	<code>logrotate -d /etc/logrotate.conf</code>	Test logrotate configuration.	<code>logrotate -d /etc/logrotate.conf</code>
296	<code>cut -d ' ' -f1 <file></code>	Extract first field from file.	<code>cut -d ' ' -f1 access.log</code>
297	<code>sort <file></code>	Sort lines in file.	<code>sort access.log</code>
298	<code>uniq <file></code>	Remove duplicate lines.	<code>uniq access.log</code>
299	<code>wc -l <file></code>	Count number of lines.	<code>wc -l access.log</code>
300	<code>rm -rf /var/log/*.gz</code>	Delete old compressed logs.	<code>rm -rf /var/log/*.gz</code>

COMMON LOG LOCATIONS

- `/var/log/syslog` → System logs
- `/var/log/auth.log` → Auth logs
- `/var/log/dmesg` → Kernel logs
- `/var/log/kern.log` → Kernel logs
- `/var/log/boot.log` → Boot logs
- `/var/log/nginx/` → Nginx logs
- `/var/log/apache2/` → Apache logs
- `/var/log/secure` → Secure logs

LOG LEVELS (`journalctl -p`)

- 0 emerg → System is unusable
- 1 alert → Action must be taken
- 2 crit → Critical conditions
- 3 err → Error conditions
- 4 warning → Warning conditions
- 5 notice → Normal but significant
- 6 info → Informational
- 7 debug → Debug-level messages

EXAMPLES

- # Follow logs in real-time
`tail -f /var/log/syslog`
- # Show errors from journal
`journalctl -p err`
- # Logs for nginx service
`journalctl -u nginx -f`
- # Search "fail" in logs
`grep -i "fail" /var/log/app.log`

TIP



- ✓ Use `journalctl` for systemd-based systems.
- ✓ Use `grep -i` for case-insensitive search.

- ✓ Rotate logs regularly to save disk space.
- ✓ Always monitor logs for early issue detection.



FILTER LOGS BY TIME



- Show logs from today:
`journalctl --since today`
- Show logs from yesterday:
`journalctl --since yesterday`
- Show logs from custom time:
`journalctl --since "2025-06-18 10:00"`

LOG ANALYSIS COMMANDS



- Count error occurrences:
`grep -i "error" app.log | wc -l`
- Top 10 IPs from access log:
`awk '{print $1}' access.log | sort | uniq -c | sort -nr | head`
- Find slow requests (> 2s):
`awk '$NF > 2.000 {print}' access.log`

LOG ROTATION BASICS



- Logrotate compresses old logs.
- Prevents logs from filling disk.
- Standard config: `/etc/logrotate.conf`
- Create custom config in `/etc/logrotate.d/`
- Test with: `logrotate -d <config>`

REAL-TIME LOG MONITORING TOOLS



- `tail -f` → Basic real-time
- `multitail` → Multiple log files
- `lnav` → Advanced log viewer
- `glances` → System + logs
- `watch tail -n 10 file.log` → Auto refresh

BEST PRACTICES



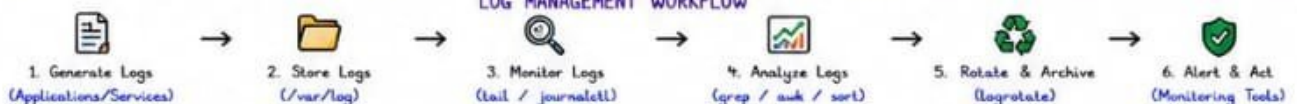
- ✓ Centralize logs (ELK, Loki, Splunk).
- ✓ Set up log alerts for critical errors.
- ✓ Keep logs for required retention only.
- ✓ Monitor disk usage of `/var/log` regularly.
- ✓ Automate log rotation and cleanup.

LOG TROUBLESHOOTING



- Logs not showing? Check permissions.
- Disk full? Rotate and compress logs.
- High log volume? Filter and archive.
- Slow system? Check log writing rate.
- Corrupted logs? Rotate and restart service.

LOG MANAGEMENT WORKFLOW



Logs tell the story. Read them. Understand them. Act on them.



FOLLOW @VERIQTAT FOR MORE DEVOPS CONTENT





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

16. PROCESS MANAGEMENT COMMANDS



Manage, monitor and control processes like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
301	ps aux	Show all running processes.	ps aux
302	ps -ef	Full format process listing.	ps -ef
303	ps -u <user>	Show processes for a specific user.	ps -u nginx
304	top	Real-time process monitor.	top
305	htop	Interactive process viewer.	htop
306	ptree	Display processes in a tree view.	ptree
307	pgrep <name>	Find process ID by name.	pgrep nginx
308	pidof <name>	Show PID of process.	pidof docker
309	kill <PID>	Terminate a process (gracefully).	kill 1234
310	kill -9 <PID>	Force kill a process.	kill -9 1234
311	pkill <name>	Kill processes by name.	pkill nginx
312	killall <name>	Kill all processes with name.	killall java
313	nice -n 10 <cmd>	Start process with nice value.	nice -n 10 python app.py
314	renice -n 5 -p <PID>	Change priority of running process.	renice -n 5 -p 1234
315	jobs	List active jobs (background).	jobs
316	bg %1	Resume job in background.	bg %1
317	fg %1	Bring job to foreground.	fg %1
318	nohup <cmd> &	Run command immune to hangup.	nohup script.sh &
319	disown %1	Remove job from shell tracking.	disown %1
320	time <cmd>	Measure command execution time.	time rsync -ah /src /dest

PROCESS STATES (ps)



- R → Running
- S → Sleeping (interruptible)
- D → Uninterruptible sleep
- Z → Zombie
- T → Stopped
- X → Dead
- I → Idle (Kernel thread)

COMMON SIGNALS (kill)



- 1 (HUP) → Hangup
- 2 (INT) → Interrupt
- 9 (KILL) → Force kill
- 15 (TERM) → Terminate (graceful)
- 18 (CONT) → Continue
- 19 (STOP) → Stop

EXAMPLES



- # Show all processes
ps aux
- # Find and kill process
pgrep nginx
kill 1234
- # Monitor system
top
- # Run in background immune to logout
nohup long-job.sh &

TIP



- ✓ Use top for a quick overview.
- ✓ Use htop for interactive monitoring.
- ✓ Use pgrep + kill for safe automation.

- ✓ Avoid kill -9 unless the process is unresponsive.
- ✓ Use nice/renice to manage CPU priority.



TOP COMMAND SHORTCUTS



- P → Sort by CPU
- M → Sort by Memory
- k → Kill a process
- r → Renice a process
- h → Help
- q → Quit

HTOP SHORTCUTS



- F2 → Setup
- F3 → Search
- F4 → Filter
- F5 → Tree view
- F6 → Sort by column
- F10 → Quit

PROCESS PRIORITY (nice)



- -20 (High priority)
 - 0 (Default)
 - 19 (Low priority)
- Lower value = Higher priority
Higher value = Lower priority

BACKGROUND JOBS



- Command & → Run in background
- Ctrl + Z → Pause job
- bg %job → Resume in bg
- fg %job → Bring to foreground
- jobs → List jobs

MONITORING TOOLS



- top → Real-time process monitor
- htop → Enhanced top (interactive)
- atop → Advanced system & process monitor
- glances → Cross-platform monitoring tool
- nmon → Performance monitoring (IBM)
- pidstat → Per-process stats (sysstat)

USEFUL ONE-LINERS



```
ps aux --sort=-%cpu | head -n 10 → Top 10 CPU
ps aux --sort=-%mem | head -n 10 → Top 10 Memory
ps -eo pid,ppid,cmd,xmem,%cpu --sort=-%cpu | head
→ Custom process list
watch -n 2 'ps aux --sort=-%cpu | head'
→ Auto refresh
```

PROCESS TROUBLESHOOTING



- High CPU usage? → Check top/htop
- Zombie process? → Check ps aux
- Process not stopping? → Use kill -9
- Unknown process? → Check logs & service
- Many processes? → Optimize or scale

PROCESS MANAGEMENT WORKFLOW



★ A well-managed process is a well-performing system. Monitor. Prioritize. Optimize. Repeat.

FOLLOW @VERIQTA FOR MORE DEVOPS CONTENT



LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

17. USER & GROUP MANAGEMENT COMMANDS



Manage users, groups and permissions like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
321	whoami	Show current logged in user.	whoami
322	id	Show user and group IDs.	id
323	users	Show logged in users.	users
324	who	Show who is logged in.	who
325	last	Show last logged in users.	last
326	adduser <user>	Add a new user.	sudo adduser john
327	useradd <user>	Create a new user (low level).	sudo useradd devops
328	usermod -aG <group> <user>	Add user to group.	sudo usermod -aG docker john
329	passwd <user>	Change user password.	sudo passwd john
330	deluser <user>	Remove a user.	sudo deluser john
331	userdel <user>	Delete a user (low level).	sudo userdel -r john
332	groupadd <group>	Add a new group.	sudo groupadd devops
333	groupdel <group>	Delete a group.	sudo groupdel devops
334	groups <user>	Show groups of user.	groups john
335	chage -l <user>	Show password aging info.	sudo chage -l john
336	chage -M <days> <user>	Set max password age.	sudo chage -M 90 john
337	su - <user>	Switch to another user.	su - john
338	sudo <command>	Run command as superuser.	sudo apt update
339	visudo	Edit sudoers file safely.	sudo visudo
340	newgrp <group>	Switch primary group.	newgrp docker

USER TYPES



- root → Superuser
- system user → Service accounts
- regular user → Human users

IMPORTANT FILES



- /etc/passwd → User accounts
- /etc/shadow → Password hashes
- /etc/group → Groups
- /etc/sudoers → Sudo permissions

COMMON GROUPS



- sudo → Admin users
- docker → Docker access
- www-data → Web server
- adm → System logs
- dev → Device access

EXAMPLES



- # Add user john
sudo adduser john
- # Add john to docker group
sudo usermod -aG docker john
- # Check john's groups
groups john
- # Run command as root
sudo systemctl restart nginx

TIP



- ✓ Always use **sudo** for admin tasks.
- ✓ Use **visudo** to edit sudoers file.

- ✓ Never share passwords.
- ✓ Regularly review user & group access.



ADD USER WORKFLOW



1. adduser john
2. passwd john
3. usermod -aG sudo john
4. groups john
5. Test login

DELETE USER WORKFLOW



1. deluser john
2. userdel -r john (remove home)
3. Check /etc/passwd
4. Check /home

CHECK USER INFO



- id john → UID, GID, groups
- chage -l john → Password aging
- last john → Login history
- whoami → Current user

SUDO BEST PRACTICES



- ✓ Grant minimum required access.
- ✓ Use specific commands in sudoers.
- ✓ Audit sudo access regularly.
- ✓ Never allow full root access.

/etc/sudoers SYNTAX



```
# Allow user to run specific commands
john ALL=(ALL) /bin/systemctl restart nginx

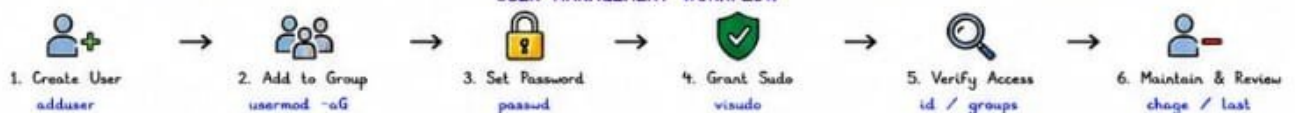
# Allow user full sudo access
john ALL=(ALL) ALL
```

USER SECURITY TIPS



- ✓ Use strong passwords.
- ✓ Disable root login via SSH.
- ✓ Use key-based authentication.
- ✓ Lock unused accounts.

USER MANAGEMENT WORKFLOW



★ Good access control keeps your system secure. Manage users wisely. Stay secure. 🏆



LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

18. NETWORK MANAGEMENT COMMANDS



Configure, troubleshoot and monitor networks like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
341	<code>ip addr</code>	Show IP addresses.	<code>ip addr</code>
342	<code>ip a</code>	Show IP addresses (shortcut).	<code>ip a</code>
343	<code>ip route</code>	Show routing table.	<code>ip route</code>
344	<code>ip r</code>	Show routing table (shortcut).	<code>ip r</code>
345	<code>ip link</code>	Show network interfaces.	<code>ip link</code>
346	<code>ip l</code>	Show interfaces (shortcut).	<code>ip l</code>
347	<code>ping <host></code>	Test connectivity to host.	<code>ping google.com</code>
348	<code>ping -c 4 <host></code>	Send 4 ping packets.	<code>ping -c 4 8.8.8.8</code>
349	<code>tracert <host></code>	Trace network path to host.	<code>tracert google.com</code>
350	<code>mtr <host></code>	My traceroute (better traceroute).	<code>mtr google.com</code>
351	<code>ss -tulnp</code>	Show listening ports and processes.	<code>ss -tulnp</code>
352	<code>netstat -tulnp</code>	Show TCP connections.	<code>netstat -tulnp</code>
353	<code>netstat -ltnp</code>	Show listening ports (Legacy).	<code>netstat -ltnp</code>
354	<code>curl <url></code>	Transfer data from or to server.	<code>curl https://example.com</code>
356	<code>curl -I <url></code>	Fetch headers only.	<code>curl -I https://google.com</code>
357	<code>wget <url></code>	Download file from web.	<code>wget https://file.com/file.zip</code>
358	<code>dig <domain></code>	DNS lookup tool.	<code>dig example.com</code>
359	<code>nslookup <domain></code>	DNS lookup (Legacy).	<code>nslookup example.com</code>
360	<code>host <domain></code>	Simple DNS lookup.	<code>host example.com</code>

NETWORK INTERFACE FILES



- `/etc/network/interfaces` → Debian/Ubuntu
- `/etc/sysconfig/network-scripts/ifcfg-*` → RHEL/CentOS
- `/etc/hosts` → Hostname mapping
- `/etc/resolv.conf` → DNS config

IP ADDRESS TYPES



- IPv4 → 192.168.1.1
- IPv6 → fe80::1
- Private IPv4 Ranges:
 - 10.0.0.0/8
 - 172.16.0.0/12
 - 192.168.0.0/16

COMMON PORTS



- 22 → SSH
- 80 → HTTP
- 443 → HTTPS
- 3306 → MySQL
- 5432 → PostgreSQL
- 6379 → Redis

EXAMPLES



- # Check IP address
`ip addr`
- # Ping a server
`ping -c 4 google.com`
- # Trace route
`tracert google.com`
- # Check open ports
`ss -tulnp`
- # Download a file
`wget https://file.com/app.tar.gz`

TIP



- Use `ip a` instead of `ifconfig` (deprecated).
- Use `ss` instead of `netstat` (Faster & modern).

- Always test connectivity after network changes.
- Keep `/etc/hosts` updated for internal services.



IP COMMAND ESSENTIALS



- `ip addr show` → Show IPs
- `ip link set eth0 up` → Enable interface
- `ip link set eth0 down` → Disable interface
- `ip route add default via 192.168.1.1`
- `ip route del default via 192.168.1.1`

FIREWALL & PORT CHECKS



- `sudo ufw status` → UFW status (Ubuntu)
- `sudo ufw allow 22` → Allow port 22
- `sudo firewall-cmd --list-ports` → RHEL
- `sudo iptables -L -n` → iptables rules
- `sudo nft list ruleset` → nftables rules

DNS & NAME RESOLUTION



- `cat /etc/resolv.conf` → DNS servers
- `dig +short example.com` → IP only
- `nslookup -type=mx example.com` → Mail exch.
- `host -l A example.com` → IPv4 lookup

NETWORK TROUBLESHOOTING



- `ping -c 4 8.8.8.8` → Test external
- `ping -c 4 localhost` → Test local
- `tracert -n 8.8.8.8` → Trace route
- `mtr -rw 8.8.8.8` → Real-time traceroute
- `ss -s` → Socket summary

DOWNLOAD & TRANSFER TOOLS



- `curl -O <url>` → Download file
- `curl -u user:pass <url>` → Auth download
- `wget -c <url>` → Resume download
- `scp file user@host:/path/` → Secure copy
- `rsync -avz src/ user@host:/dest/` → Sync

MONITORING NETWORK USAGE



- `iftop` → Real-time bandwidth (install)
- `nethogs` → Per-process bandwidth
- `ip -s link` → Interface stats
- `sar -n DEV 1` → Network stats
- `vnstat` → Network traffic monitor

NETWORK TROUBLESHOOTING WORKFLOW

1. Check Interface
`ip addr`2. Check Connectivity
`ping -c 4 host`3. Trace Route
`tracert host`4. Check Open Ports
`ss -tulnp`5. Check DNS
`dig host`6. Monitor Traffic
`iftop / nethogs`7. Fix & Verify
Test again

A strong network. A strong system. Monitor. Troubleshoot. Optimize. Repeat.



FOLLOW @VERIQTA FOR MORE DEVOPS CONTENT





LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

19. PACKAGE MANAGEMENT COMMANDS

Install, update and manage software packages like a **PRO**.

#	COMMAND	DESCRIPTION	EXAMPLE
361	<code>apt update</code>	Update package index.	<code>sudo apt update</code>
362	<code>apt upgrade</code>	Upgrade all packages.	<code>sudo apt upgrade</code>
263	<code>apt install <pkg></code>	Install a package.	<code>sudo apt install nginx</code>
364	<code>apt remove <pkg></code>	Remove a package.	<code>sudo apt remove nginx</code>
365	<code>apt purge <pkg></code>	Remove package + config files.	<code>sudo apt purge nginx</code>
366	<code>apt search <pkg></code>	Search for a package.	<code>apt search docker</code>
367	<code>apt show <pkg></code>	Show package details.	<code>apt show nginx</code>
368	<code>apt list --installed</code>	List installed packages.	<code>apt list --installed</code>
369	<code>apt autoremove</code>	Remove unused packages.	<code>sudo apt autoremove</code>
370	<code>apt clean</code>	Clean cache.	<code>sudo apt clean</code>
371	<code>yum install <pkg></code>	Install package (RHEL/CentOS).	<code>sudo yum install httpd</code>
372	<code>yum remove <pkg></code>	Remove package (RHEL/CentOS).	<code>sudo yum remove httpd</code>
373	<code>dnf install <pkg></code>	Install package (Fedora).	<code>sudo dnf install nginx</code>
374	<code>dnf remove <pkg></code>	Remove package (Fedora).	<code>sudo dnf remove nginx</code>
375	<code>dnf update</code>	Update all packages.	<code>sudo dnf update</code>
376	<code>zypper install <pkg></code>	Install package (openSUSE).	<code>sudo zypper install nginx</code>
377	<code>zypper remove <pkg></code>	Remove package (openSUSE).	<code>sudo zypper remove nginx</code>
378	<code>rpm -qa</code>	List all installed RPM packages.	<code>rpm -qa</code>
379	<code>rpm -qi <pkg></code>	Show RPM package info.	<code>rpm -qi httpd</code>
380	<code>dpkg -l</code>	List installed Debian packages.	<code>dpkg -l</code>

PACKAGE MANAGERS CHEAT SHEET

- Debian/Ubuntu → `apt`
- RHEL/CentOS → `yum / dnf`
- Fedora → `dnf`
- openSUSE → `zypper`
- Arch Linux → `pacman`
- Alpine Linux → `apk`

USEFUL OPTIONS

- `-y / --assume-yes` → Auto yes
- `-q / --quiet` → Quiet mode
- `--dry-run` → Simulate only
- `--no-install-recommends` → Skip extras

CHECK PACKAGE OWNERSHIP

- `dpkg -S /path/to/file` (Debian)
- `rpm -qf /path/to/file` (RHEL)
- `dnf provides */nginx` (Find provider)

EXAMPLES

- ```
Update and upgrade system
sudo apt update && sudo apt upgrade -y

Install nginx
sudo apt install nginx -y

Remove nginx completely
sudo apt purge nginx -y

Clean up unused packages
sudo apt autoremove -y && sudo apt clean
```

## TIP



- ✓ Always update package index before installing.
- ✓ Use `autoremove` to keep system clean.

- ✓ Prefer `dnf` over `yum` on modern RHEL based distros.
- ✓ Check package size before installing: `apt show <pkg>`



## APT COMMAND FLOW



1. `apt update` → Update index
2. `apt upgrade` → Upgrade packages
3. `apt install <pkg>` → Install package
4. `apt autoremove` → Remove unused
5. `apt clean` → Clean cache

## RHEL/FEDORA (DNF) FLOW



1. `dnf update` → Update index
2. `dnf upgrade` → Upgrade packages
3. `dnf install <pkg>` → Install package
4. `dnf autoremove` → Remove unused
5. `dnf clean all` → Clean cache

## PACMAN (ARCH) QUICKIES



- ```
pacman -Syu → Update system
pacman -S <pkg> → Install package
pacman -R <pkg> → Remove package
pacman -Qs <pkg> → Search package
pacman -Sc → Clean cache
```

FIND OUTDATED PACKAGES



- `apt list --upgradable` (Debian/Ubuntu)
- `dnf check-update` (Fedora/RHEL)
- `zypper list-updates` (openSUSE)
- `pacman -Qu` (Arch Linux)

VERIFY PACKAGE & VERSION



- `dpkg -s <pkg>` (Detailed status)
- `rpm -qi <pkg>` (RPM info)
- `dnf info <pkg>` (Package info)
- `apt policy <pkg>` (Version policy)

COMMON TASKS



- Rollback package → Use specific version
- List repo sources → `/etc/apt/sources.list`
`/etc/yum/repos.d/`
- Check repo status → `dnf repolist`
- Download package → `apt download <pkg>`

PACKAGE MANAGEMENT WORKFLOW



1. Search Package
`apt search nginx`

2. Update Index
`apt update`

3. Install Package
`apt install nginx`

4. Verify Install
`systemctl status nginx`

5. Clean System
`apt autoremove && apt clean`

Keep your system updated. Keep it clean. Keep it secure. **Update. Upgrade. Maintain.**

FOLLOW @VERIQTAT FOR MORE DEVOPS CONTENT



LINUX PRODUCTION HANDBOOK

★ Top 500 Linux Commands Every DevOps Engineer Should Know ★

The Ultimate Linux Notebook for DevOps, Cloud & Platform Engineers

20. SYSTEM MAINTENANCE & HOUSEKEEPING COMMANDS



Keep systems healthy, optimized and running smoothly like a PRO.

#	COMMAND	DESCRIPTION	EXAMPLE
381	df -h	Show disk space usage.	df -h
382	du -sh <dir>	Show folder size.	du -sh /var/log
383	du -h --max-depth=1	Show top-level folder sizes.	du -h --max-depth=1 /
384	ncdu	Interactive disk usage analyzer.	ncdu /var
385	free -h	Show memory usage.	free -h
386	top	Real-time system monitor.	top
387	htop	Enhanced system monitor.	htop
388	uptime	Show system uptime and load.	uptime
389	uname -a	Show kernel and system info.	uname -a
390	cat /proc/cpuinfo	Show CPU information.	cat /proc/cpuinfo
391	cat /proc/meminfo	Show memory information.	cat /proc/meminfo
392	iostat -xz 1	Monitor disk I/O statistics.	iostat -xz 1
393	vmstat 1	Virtual memory statistics.	vmstat 1
394	sar -u 1 5	CPU usage report.	sar -u 1 5
395	sar -r 1 5	Memory usage report.	sar -r 1 5
396	journalctl --vacuum-time=7d	Clean logs older than 7 days.	journalctl --vacuum-time=7d
397	logrotate -f /etc/logrotate.conf	Force log rotation.	logrotate -f /etc/logrotate.conf
398	sync	Flush filesystem buffers.	sync
399	shutdown -r now	Reboot system now.	shutdown -r now
400	shutdown -h now	Shutdown system now.	shutdown -h now

DISK CLEANUP COMMANDS



- apt clean → Clean apt cache
- yum clean all → Clean yum cache
- dnf clean all → Clean dnf cache
- journalctl --vacuum-size=500M → Limit journal size
- rm -rf /tmp/* → Clear temp files
- find / -type f -name "*.log" -delete → Delete old logs

SYSTEM HEALTH CHECK



- uptime → Check uptime
- top / htop → Monitor processes
- df -h → Check disk space
- free -h → Check memory
- iostat -xz 1 → Check disk I/O
- journalctl -xe → Check errors

EXAMPLES



- # Check disk usage
df -h
- # Find large directories
du -h --max-depth=1 /
- # Clean old logs
journalctl --vacuum-time=7d
- # Monitor system
htop
- # Reboot system
shutdown -r now

TIP



- ✓ Monitor systems regularly.
- ✓ Automate cleanups with cron jobs.

- ✓ Keep logs under control.
- ✓ Prevent issues before they happen.



AUTOMATE CLEANUP (CRON JOB)



Add cleanup tasks to crontab:

```
0 2 * * * /usr/bin/journalctl --vacuum-time=7d
0 3 * * * /usr/bin/logrotate -f /etc/logrotate.conf
*/30 * * * * /usr/bin/apt autoremove -y
```

LOG MANAGEMENT TIPS



- ✓ Rotate logs regularly with logrotate.
- ✓ Compress old logs to save space.
- ✓ Monitor log size with du -sh /var/log.
- ✓ Use centralized logging (ELK/Loki/Splunk).

PERFORMANCE TUNING BASICS



- ✓ Disable unnecessary services.
- ✓ Use latest stable kernel.
- ✓ Monitor resource usage.
- ✓ Optimize disk I/O and network.

USEFUL ALIASES



```
alias ll='ls -lF'
alias la='ls -A'
alias lx='ls -CF'
alias top='htop'
alias update='sudo apt update && sudo apt upgrade -y'
```

SYSTEM INFO COMMANDS



- hostnamectl → System hostname info
- lscpu → CPU architecture info
- lsblk → Block devices info
- lsusb → USB devices info
- lshw → Hardware info (detailed)

HOUSEKEEPING CHECKLIST



- ✓ Keep system updated
- ✓ Monitor disk, memory & CPU
- ✓ Rotate and clean logs
- ✓ Remove unused packages
- ✓ Backup important data

SYSTEM MAINTENANCE WORKFLOW

1. Monitor System
top / htop / df -h2. Analyze Usage
du / iostat / free -h3. Clean & Optimize
cleanup commands4. Automate Tasks
cron / logrotate5. Verify Health
uptime / systemctl6. Backup & Protect
rsync / backup tools

Maintain today. Prevent tomorrow. Automate everything.



FOLLOW @VERIQTAT FOR MORE DEVOPS CONTENT

